Nombre: Sebastián Giraldo Franco.

Taller Databricks

1. Almacenar y consultar datos en el catálogo

Preguntas de investigación

- ¿Qué es Unity Catalog en Databricks?

R/ Unity Catalog es una herramienta de Databricks que permite organizar, administrar y controlar el acceso a los datos dentro de una plataforma. Su función principal es centralizar la gestión de los catálogos, esquemas y tablas, facilitando que diferentes usuarios puedan consultar y utilizar la información de manera segura.

Además, permite establecer permisos de acceso y mantener la trazabilidad de los datos, lo que ayuda a conocer de dónde proviene la información y cómo se transforma durante los procesos de ingeniería de datos.

- ¿Qué diferencia hay entre un DataFrame de Spark y una tabla Delta?

R/ Un DataFrame de Spark es una estructura de datos organizada en filas y columnas que permite procesar y transformar información mediante operaciones como filtros, consultas y agrupaciones. Se utiliza principalmente para trabajar con datos durante la ejecución de un programa.

Por otro lado, una tabla Delta permite almacenar los datos de manera persistente utilizando el formato Delta Lake. Además, ofrece características como el control de versiones, la consistencia de las transacciones y la posibilidad de actualizar o eliminar registros de forma confiable.

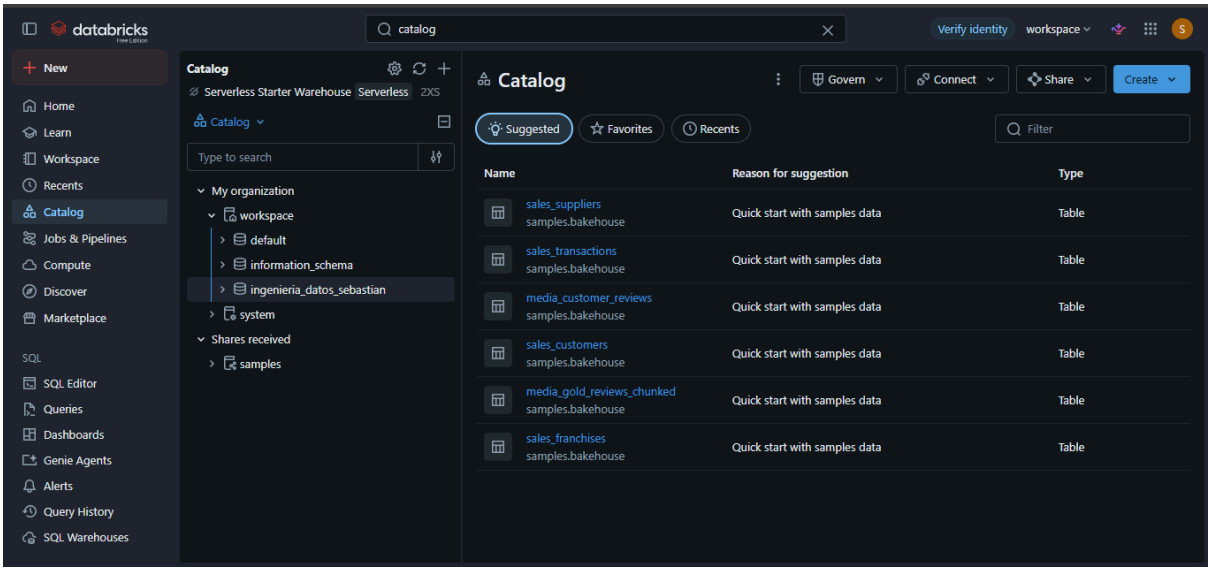
La principal diferencia es que el DataFrame se utiliza para procesar los datos, mientras que la tabla Delta permite almacenarlos y administrarlos para consultarlos posteriormente.

Fuentes:

- Databricks. What is Unity Catalog? Documentación oficial de Unity Catalog
- Apache Spark. PySpark DataFrame. Documentación oficial de DataFrame
- Databricks. Tutorial: Create and manage Delta Lake tables. Documentación oficial de tablas Delta

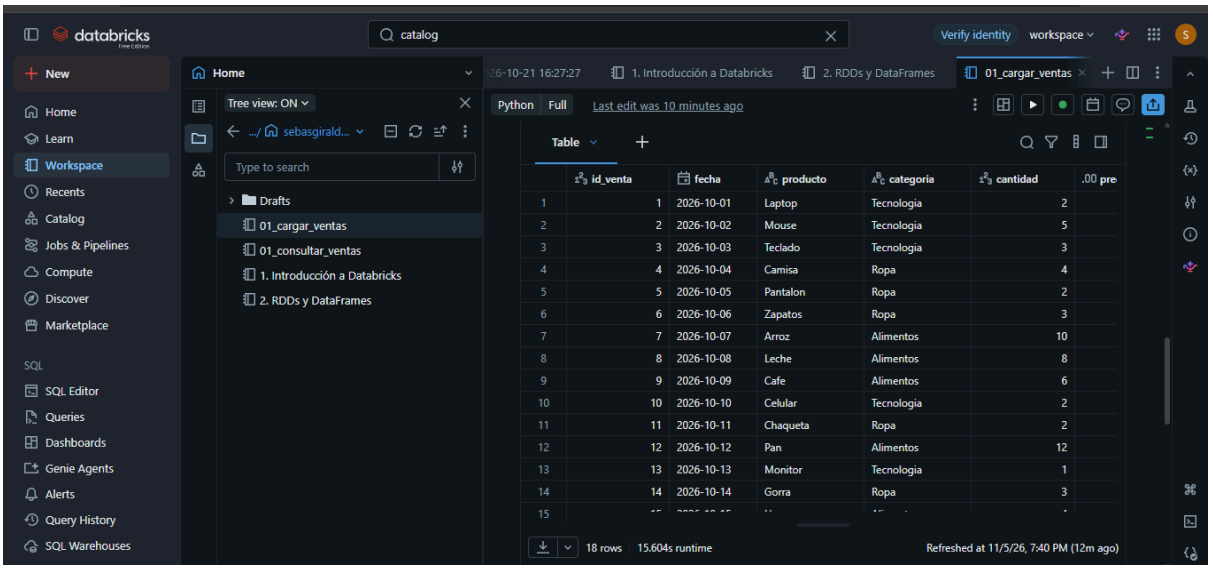
Evidencia 1. Creación del esquema en Unity Catalog

Se creó el esquema ingenieria_datos_sebastian dentro del catálogo workspace de Databricks, con el propósito de organizar y almacenar las tablas utilizadas durante el desarrollo del taller.



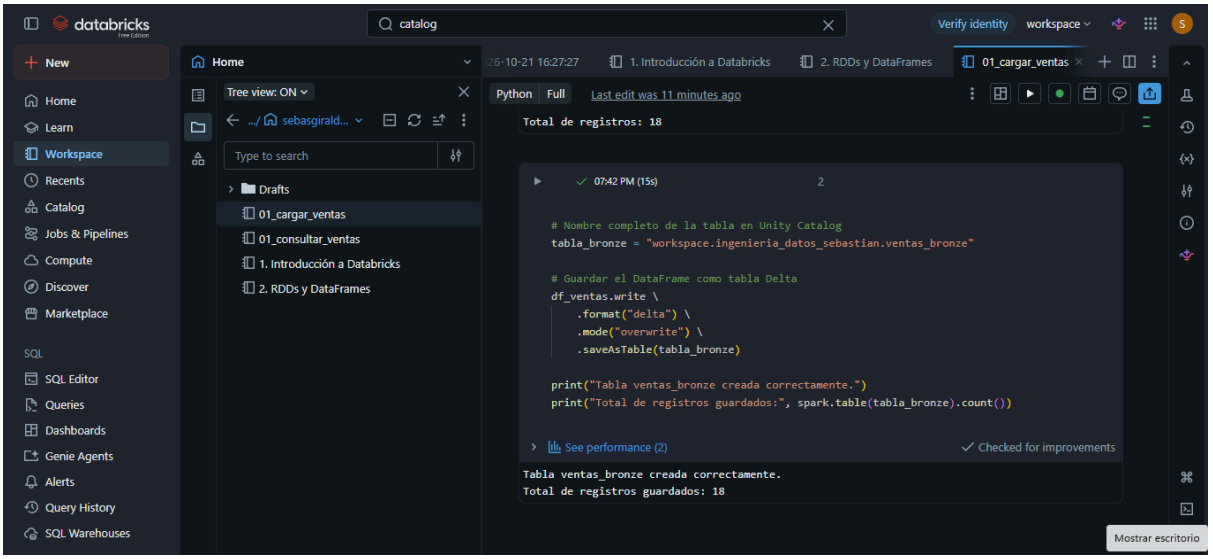
Evidencia 2. Construcción del DataFrame de ventas

Se construyó un DataFrame de Spark con 18 registros de ventas y seis columnas.



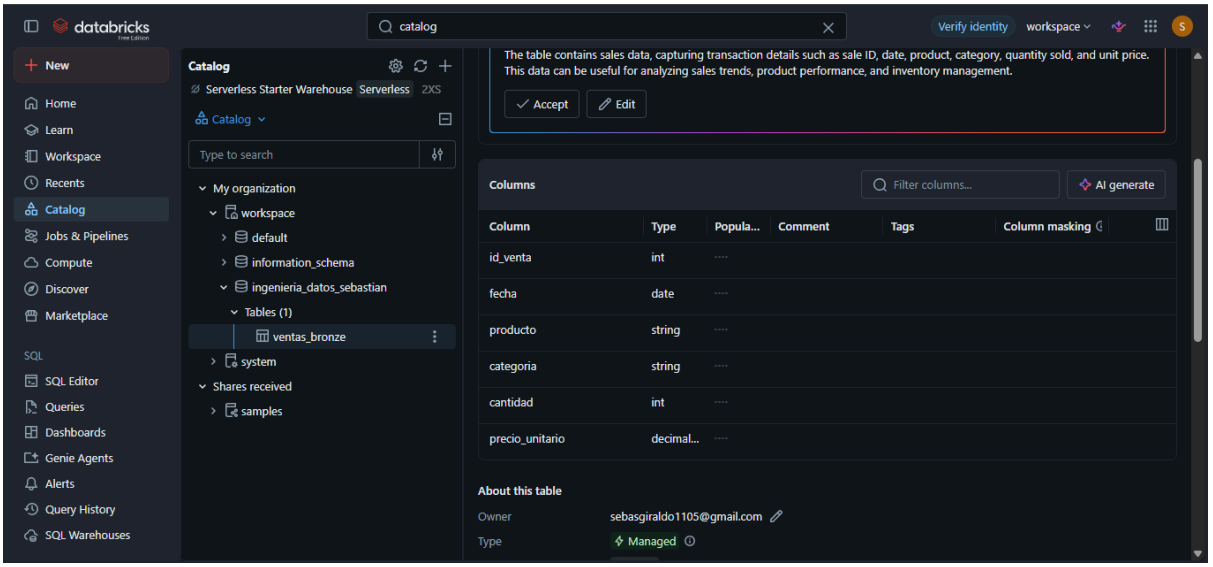
Evidencia 3. Almacenamiento de la tabla Delta

Se almacenaron los 18 registros del DataFrame en una tabla Delta denominada ventas_bronze, utilizando el esquema previamente creado. La operación se realizó mediante PySpark y se verificó que la tabla conservara la totalidad de los registros originales.



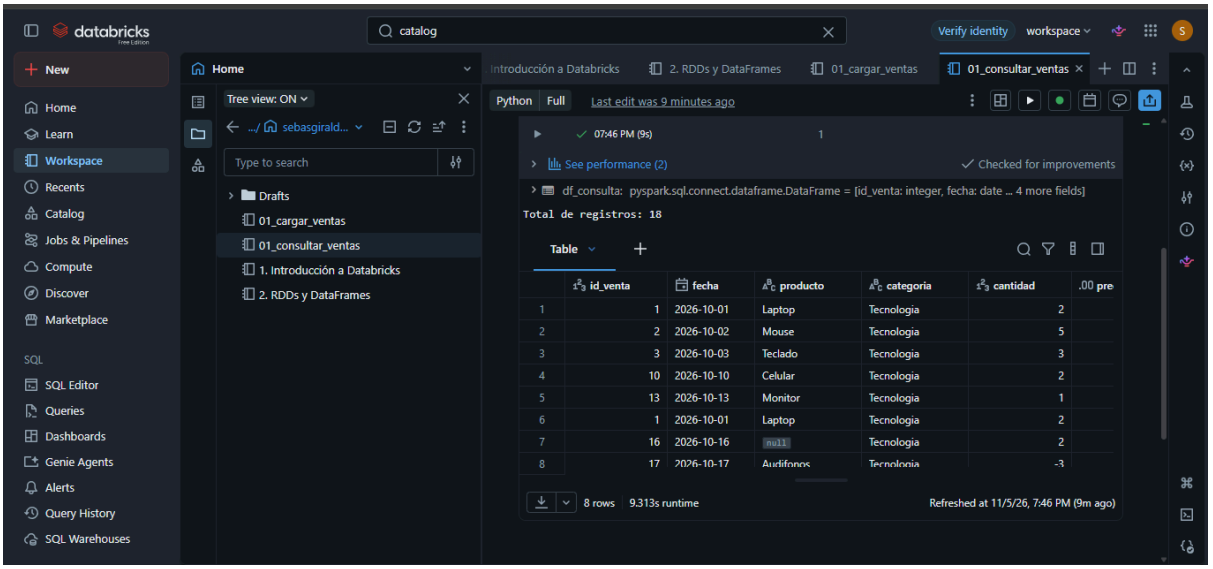
Evidencia 4. Estructura de la tabla en Unity Catalog

Se verificó la estructura de la tabla ventas_bronze desde el explorador de Unity Catalog. Se identificaron seis columnas con tipos de datos INT, DATE, STRING y DECIMAL.



Evidencia 5. Consulta de ventas desde otro notebook

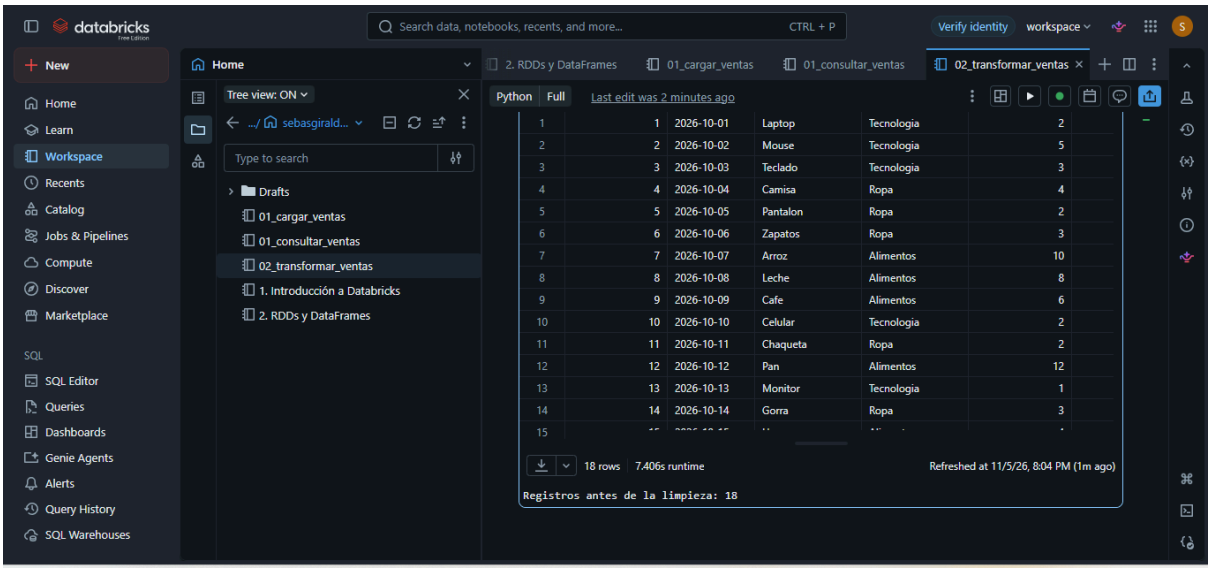
Desde el notebook 01_consultar_ventas se accedió a la tabla ventas_bronze utilizando su nombre completo en Unity Catalog. La consulta permitió verificar un total de 18 registros y obtener 8 ventas correspondientes a la categoría Tecnología, incluyendo los registros con errores intencionales.



2. Transformar datos y aplicar reglas de calidad

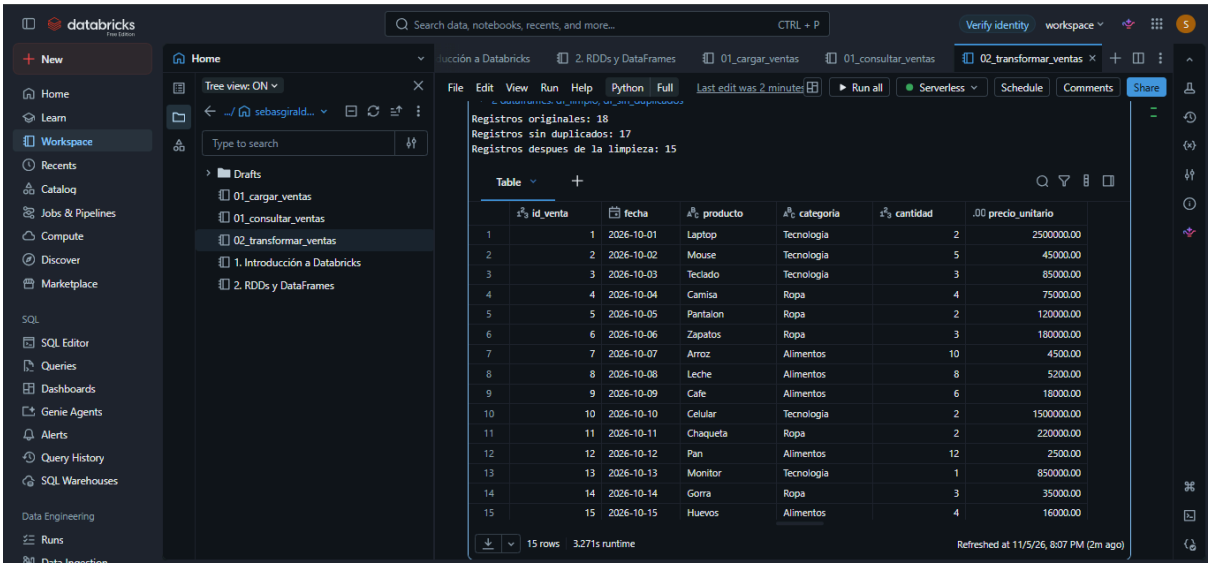
Evidencia 6. Lectura de la tabla Bronze y verificación de registros antes de la limpieza

Se realizó la lectura de la tabla `ventas_bronze` desde el notebook `02_transformar_ventas`, verificando que contiene 18 registros originales. Estos datos constituyen el punto de partida para aplicar las reglas de calidad y transformación, conservando la información original en la capa Bronze.



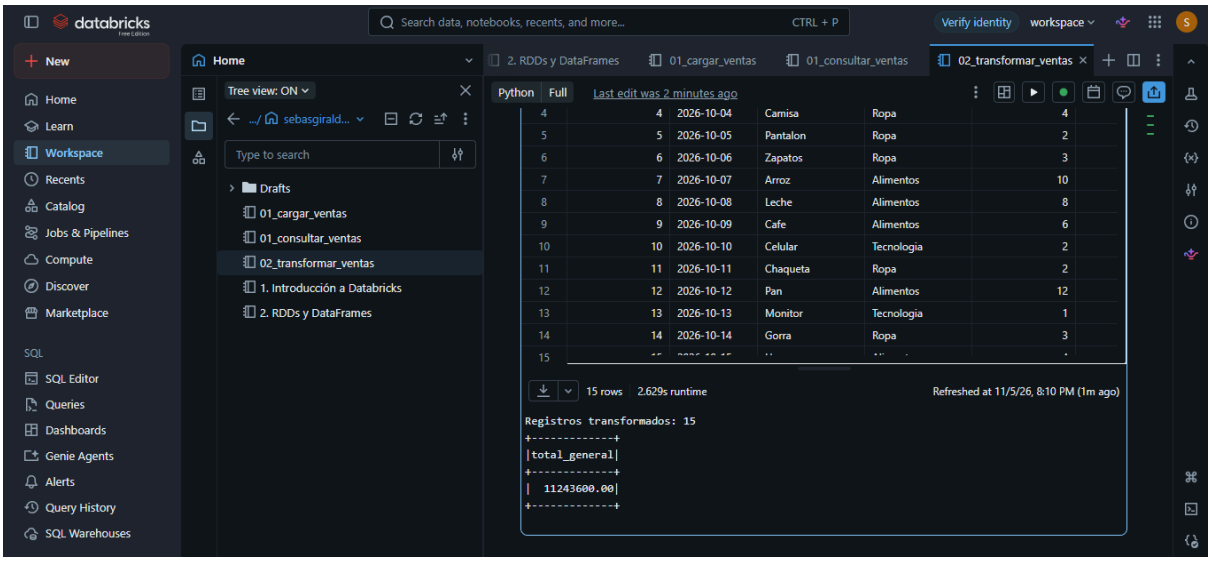
Evidencia 7. Aplicación de reglas de calidad y depuración de registros de ventas

Se aplicaron reglas de calidad sobre los 18 registros originales de la tabla ventas_bronze. Inicialmente, se eliminó un registro completamente duplicado, reduciendo el conjunto a 17 registros. Posteriormente, se descartaron las ventas con producto nulo o vacío, cantidad menor o igual a cero y precio unitario menor o igual a cero. Como resultado, se obtuvieron 15 registros válidos para continuar con las transformaciones.



Evidencia 8. Cálculo del valor total de ventas después de la limpieza

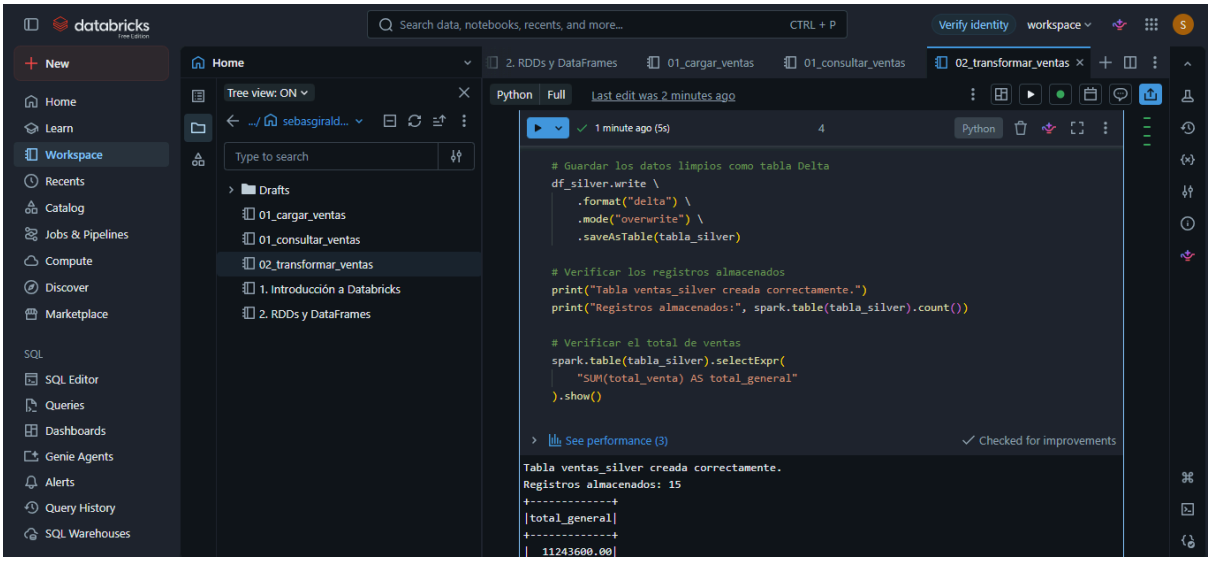
Se agregó la columna `total_venta` al DataFrame depurado, calculando el producto entre la cantidad vendida y el precio unitario de cada registro. Como resultado, se conservaron 15 ventas válidas y se obtuvo un valor total vendido de $11.243.600.



Evidencia 9. Creación y validación de la tabla Delta ventas_silver

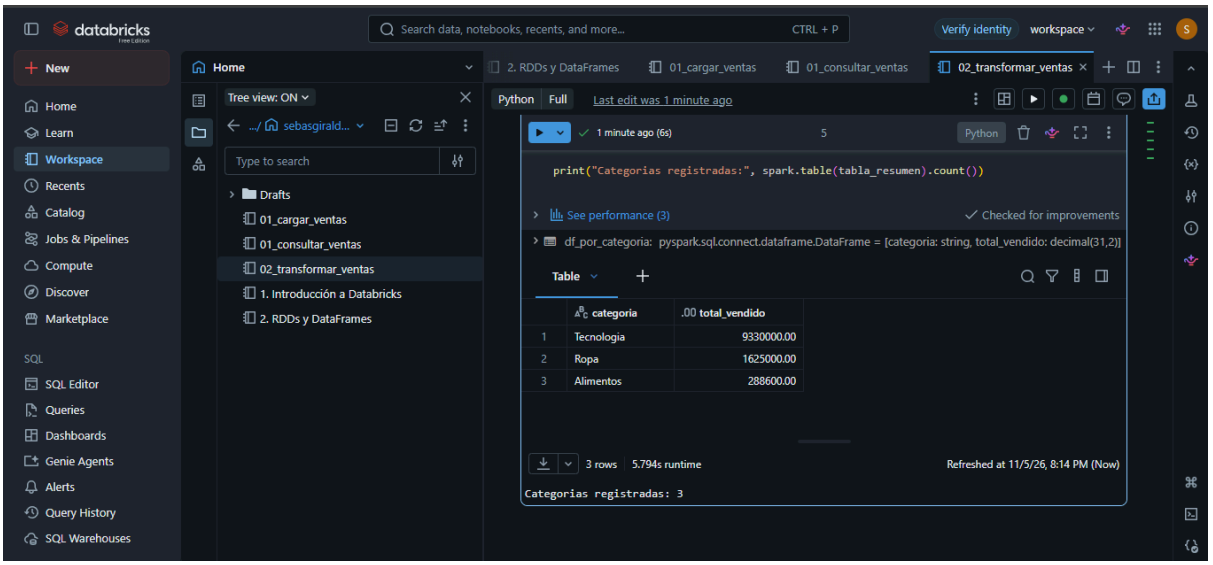
Se almacenaron los 15 registros depurados en una tabla Delta denominada ventas_silver, dentro del esquema ingenieria_datos_sebastian. Se verificó la cantidad de registros

almacenados y se confirmó un valor total de ventas de $11.243.600. Para la escritura se utilizó el modo overwrite, con el propósito de evitar la acumulación de registros al repetir el proceso.



Evidencia 10. Resumen del valor total vendido por categoría

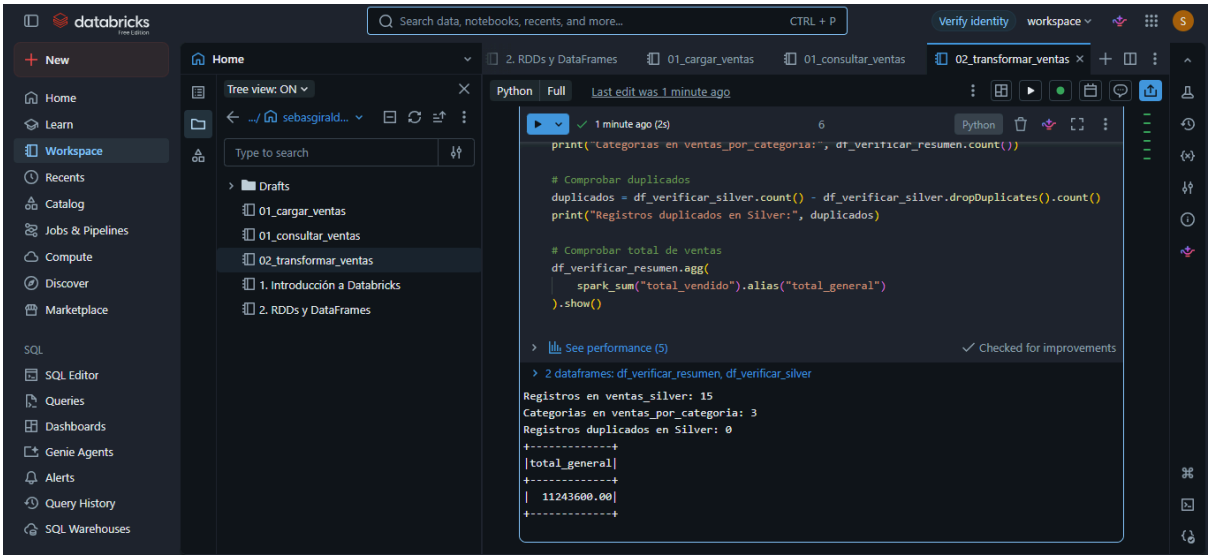
Se agruparon los registros de la tabla ventas_silver por categoría y se calculó el valor total vendido mediante funciones de agregación de PySpark. Los resultados se almacenaron en la tabla Delta ventas_por_categoria, obteniendo tres categorías: Tecnología ($9.330.000), Ropa ($1.625.000) y Alimentos ($288.600). La suma de estos valores corresponde a $11.243.600, coincidiendo con el total calculado previamente sobre los 15 registros válidos.



Evidencia 11. Verificación de la idempotencia del proceso de transformación de ventas

Se ejecutó nuevamente el notebook 02_transformar_ventas para verificar que las tablas generadas no acumularan registros duplicados. Después de repetir el proceso, la tabla ventas_silver conservó 15 registros válidos, mientras que ventas_por_categoria mantuvo tres categorías y un valor total vendido de $11.243.600.

Adicionalmente, se comprobó que la tabla Silver no contenía registros idénticos duplicados. Estos resultados evidencian que el proceso produce salidas consistentes al ejecutarse repetidamente sobre los mismos datos de origen, gracias al uso del modo de escritura overwrite.



3. Automatizar el proceso mediante un job

Preguntas de investigación

- ¿Qué es un job en Databricks y para qué sirve?

R/ Un Job en Databricks es un mecanismo que permite automatizar la ejecución de procesos de datos, como notebooks, scripts y pipelines. Su propósito es ejecutar estas actividades de manera organizada, sin necesidad de iniciarlas manualmente una por una. Además, permite configurar dependencias, programar ejecuciones y consultar los resultados o errores que se presenten durante el proceso.

- ¿Qué es una tarea y cómo se diferencia de un job?

R/ Una tarea es una actividad específica que se ejecuta dentro de un Job, como correr un notebook o realizar una transformación de datos. La diferencia es que el Job representa el proceso completo de automatización y puede contener varias tareas, mientras que cada tarea realiza una operación particular dentro de ese proceso.

- ¿Qué significa establecer dependencias entre tareas?

R/ Establecer dependencias entre tareas significa definir el orden y las condiciones bajo las cuales deben ejecutarse. Por ejemplo, una tarea de transformación puede depender de que una tarea de carga de datos termine correctamente. Esto permite mantener la secuencia lógica del proceso y evita ejecutar transformaciones cuando los datos de origen no están disponibles o la carga ha fallado.

- ¿Qué información del historial de ejecuciones permite identificar un fallo?

R/ El historial de ejecuciones permite revisar el estado de cada tarea, su duración, la hora de inicio y finalización, y los mensajes de error generados. También permite identificar qué tarea falló y consultar sus registros de ejecución para determinar la causa. Esta información facilita el diagnóstico y la corrección de problemas en los procesos automatizados.

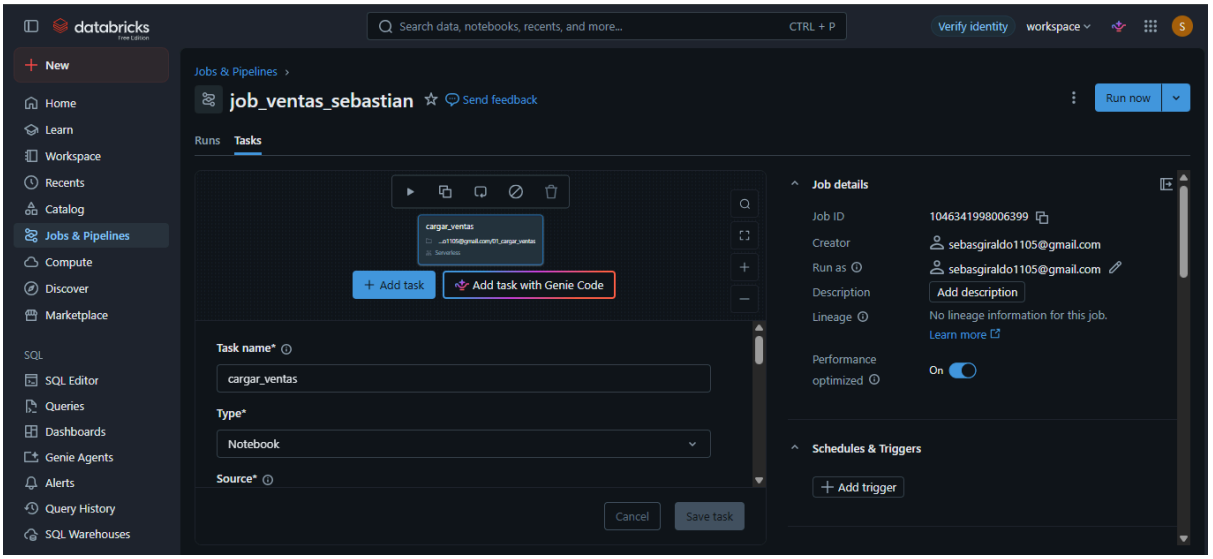
Fuentes:

- Databricks. Lakeflow Jobs: conceptos y funcionamiento
- Databricks. Configuración de tareas en Jobs
- Databricks. Monitoreo de ejecuciones de Jobs

Evidencia 12. Creación del Job y configuración de la tarea de carga de ventas en Databricks.

Se creó el Job job_ventas_sebastian mediante la herramienta Jobs & Pipelines de Databricks, con el propósito de automatizar los procesos de carga y transformación de ventas.

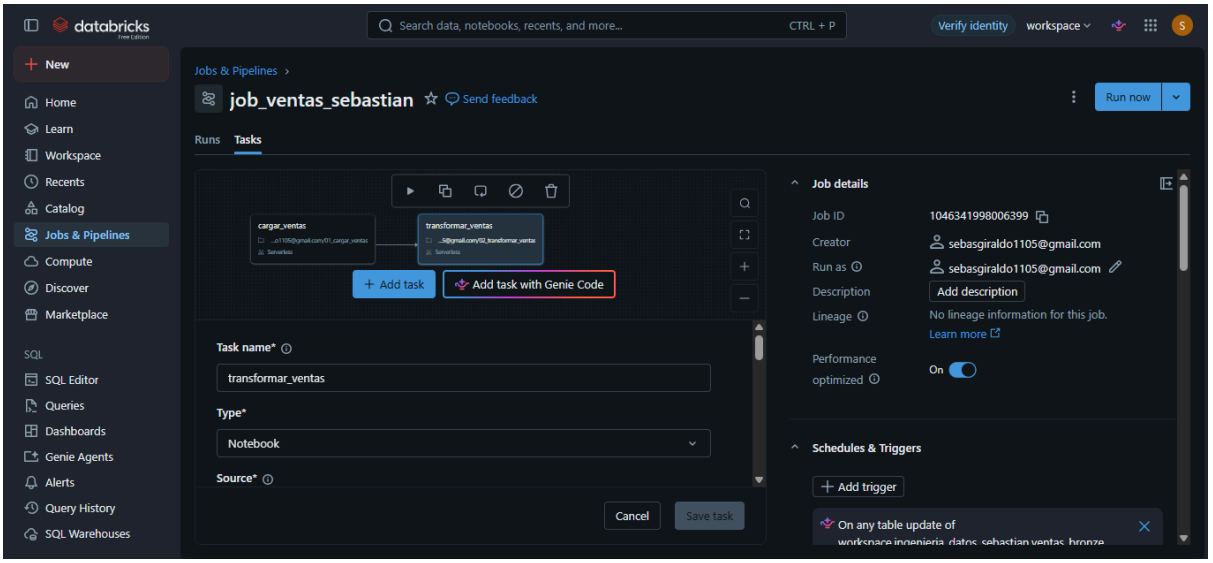
Se configuró la primera tarea, denominada cargar_ventas, asociada al notebook 01_cargar_ventas y utilizando cómputo Serverless. Esta tarea será responsable de cargar los registros originales en la tabla Delta ventas_bronze, que posteriormente servirá como fuente para el proceso de transformación.



Evidencia 13. Configuración de dependencias entre las tareas de carga y transformación de ventas.

Se configuraron dos tareas dentro del Job job_ventas_sebastian. La primera, cargar_ventas, ejecuta el notebook encargado de almacenar los registros originales en la tabla ventas_bronze. La segunda, transformar_ventas, ejecuta el notebook que aplica las reglas de calidad y genera las tablas ventas_silver y ventas_por_categoria.

Se estableció una dependencia para que la transformación únicamente se ejecute cuando la tarea de carga finalice correctamente, garantizando la secuencia lógica del proceso.

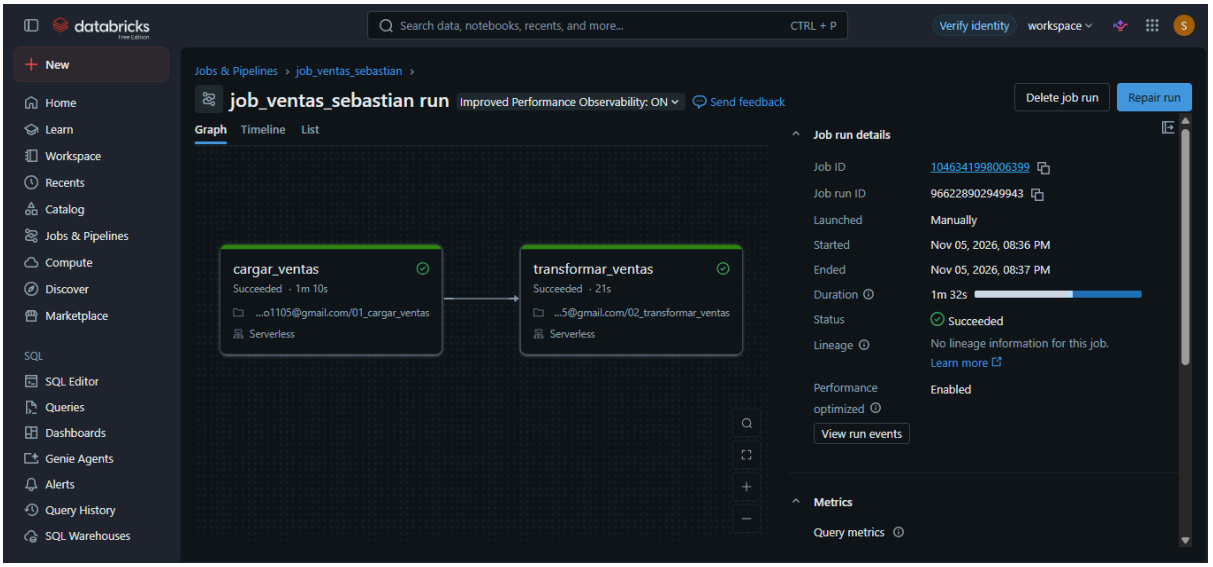


Evidencia 14. Primera ejecución exitosa del Job automatizado de carga y transformación de ventas.

Se realizó la primera ejecución manual del Job `job_ventas_sebastian`, configurado con dos tareas dependientes y cómputo Serverless.

La tarea `cargar_ventas` finalizó exitosamente en 1 minuto y 10 segundos. Posteriormente, se ejecutó `transformar_ventas`, que completó el proceso en 21 segundos.

La ejecución general finalizó con estado `Succeeded` y una duración total de 1 minuto y 32 segundos. Estos resultados permiten verificar el funcionamiento de las dependencias y la correcta automatización del flujo de carga y transformación de datos.

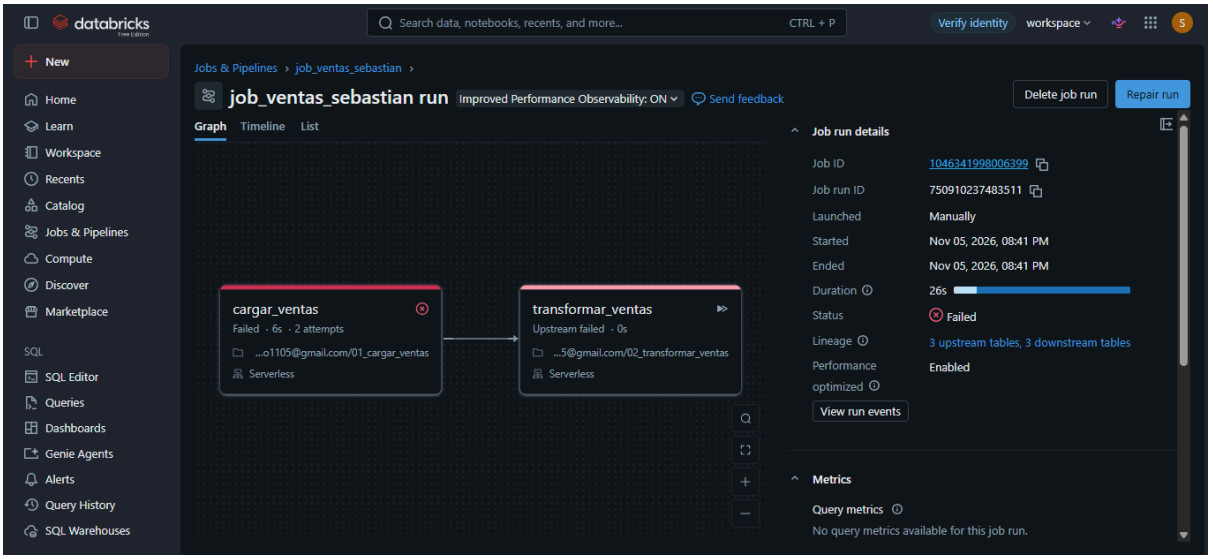


Evidencia 15. Ejecución fallida del Job por una excepción controlada en la tarea de carga.

Se realizó una segunda ejecución del Job `job_ventas_sebastian`, introduciendo deliberadamente una excepción de Python en el notebook `01_cargar_ventas`, con el objetivo de evaluar el comportamiento del sistema ante un fallo.

La tarea `cargar_ventas` terminó con estado `Failed`, después de dos intentos registrados. Como consecuencia, la tarea dependiente `transformar_ventas` presentó el estado `Upstream failed` y no fue ejecutada.

El Job finalizó con estado general `Failed` y una duración de 26 segundos. Esta prueba permitió comprobar que Databricks identifica los fallos e impide continuar con tareas dependientes cuando no se cumplen las condiciones de ejecución.

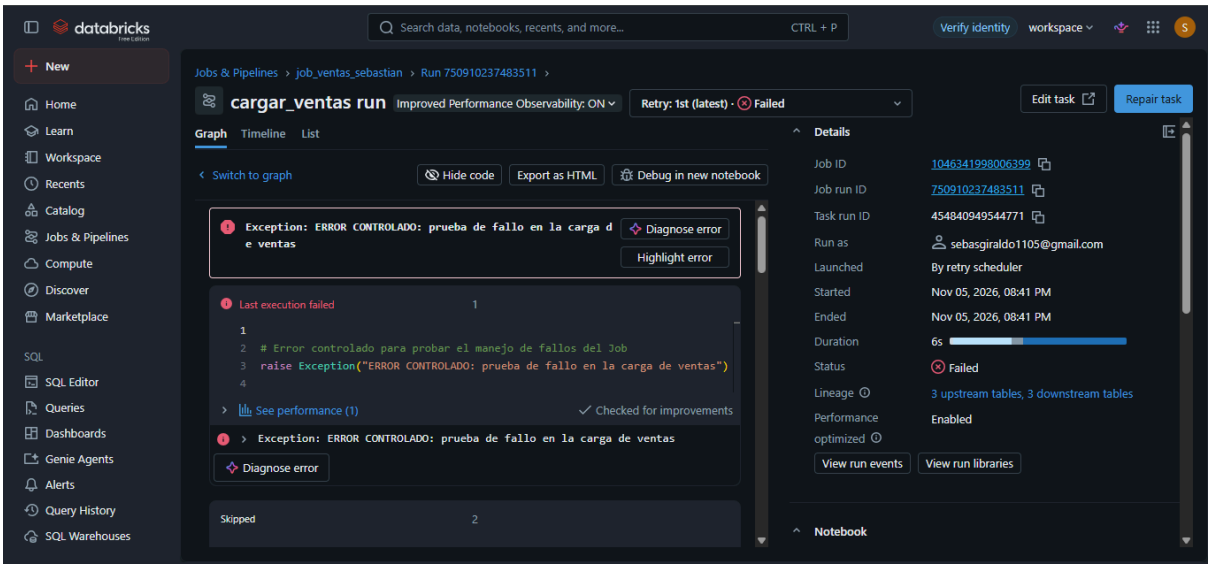


Evidencia 16. Identificación y diagnóstico de una excepción controlada en el historial de ejecución.

Al revisar los detalles de la ejecución fallida de la tarea cargar_ventas, se identificó una excepción de Python generada deliberadamente mediante la instrucción raise Exception("ERROR CONTROLADO: prueba de fallo en la carga de ventas").

El historial de Databricks permitió identificar el mensaje de error, la línea de código responsable y el estado Failed de la tarea. Debido a la dependencia configurada, la tarea transformar_ventas no se ejecutó.

Esta prueba evidenció la capacidad de Databricks para registrar fallos, facilitar su diagnóstico y controlar la ejecución de tareas dependientes dentro de un Job automatizado.

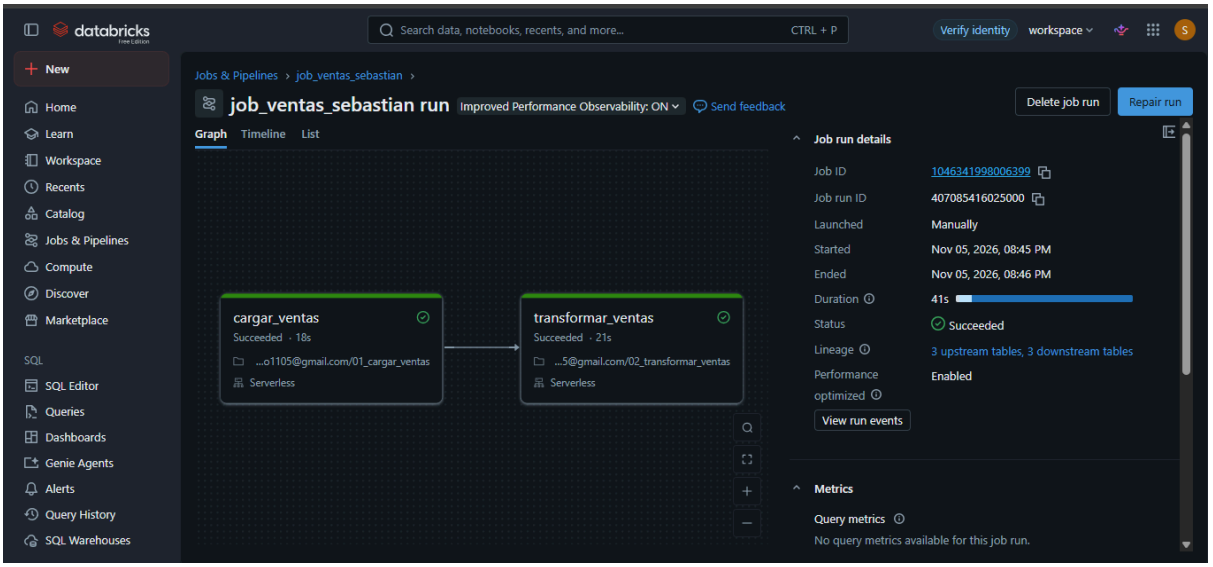


Evidencia 17. Recuperación y ejecución exitosa del Job después de corregir el fallo controlado.

Después de identificar la causa del fallo en la tarea `cargar_ventas`, se eliminó la instrucción que generaba la excepción controlada y se realizó una nueva ejecución manual del Job `job_ventas_sebastian`.

La ejecución finalizó exitosamente con estado `Succeeded` y una duración total de 41 segundos. La tarea `cargar_ventas` completó su ejecución en 18 segundos, seguida de `transformar_ventas`, que finalizó en 21 segundos.

Estos resultados demuestran que, una vez corregida la causa del error, el proceso automatizado puede ejecutarse nuevamente respetando las dependencias configuradas. También se comprobó la utilidad del historial de Databricks para identificar fallos y verificar su posterior recuperación.



4. Construir un pipeline declarativo

Preguntas de investigación

- ¿Qué es un pipeline de datos y cuáles son sus etapas habituales?

R/ Un pipeline de datos es un conjunto de procesos organizados que permiten mover, procesar y transformar información desde una o varias fuentes hasta un destino donde pueda ser consultada o analizada.

Sus etapas habituales incluyen la extracción o ingesta de datos desde las fuentes originales, el almacenamiento inicial, la limpieza y validación de la información, la transformación de los datos y su publicación para el análisis.

En Databricks, estas etapas pueden organizarse mediante la arquitectura Medallion, que utiliza tres capas: Bronze, para conservar los datos originales; Silver, para almacenar información limpia y validada; y Gold, para generar datos agregados y preparados para el análisis.

- ¿Qué significa que un pipeline sea declarativo?

R/ Un pipeline declarativo es aquel en el que se define qué conjuntos de datos se desean obtener y qué transformaciones deben aplicarse, sin necesidad de programar manualmente toda la secuencia de ejecución.

En Databricks, los pipelines declarativos permiten definir tablas y sus transformaciones mediante SQL o Python. A partir de estas definiciones, la plataforma identifica las dependencias entre los conjuntos de datos y administra su procesamiento.

Este enfoque facilita el mantenimiento del código, la organización de las transformaciones y la automatización de los flujos de datos.

- ¿En qué se diferencian un job que ejecuta notebooks y un pipeline declarativo que define conjuntos de datos?

R/ La principal diferencia está en la forma de definir y administrar el procesamiento.

Un Job que ejecuta notebooks permite automatizar tareas específicas y establecer explícitamente sus dependencias y condiciones de ejecución. Por ejemplo, primero ejecutar un notebook que carga ventas y después otro que realiza la limpieza.

En cambio, un pipeline declarativo se centra en definir los conjuntos de datos que deben producirse y las transformaciones necesarias para obtenerlos. Databricks analiza estas definiciones, identifica las dependencias y organiza su ejecución.

En conclusión, los Jobs se orientan principalmente a la orquestación de tareas, mientras que los pipelines declarativos se enfocan en la construcción y actualización de conjuntos de datos. Ambos mecanismos pueden complementarse dentro de una misma solución de ingeniería de datos.

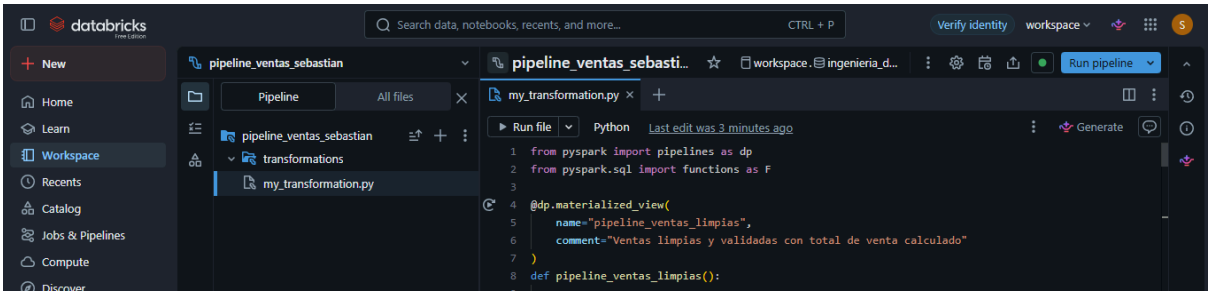
Fuentes:

- Databricks — Lakeflow: ingesta, transformación y orquestación de datos
- Databricks — Lakeflow Spark Declarative Pipelines

- Databricks — Arquitectura Medallion
- Databricks — Lakeflow Jobs

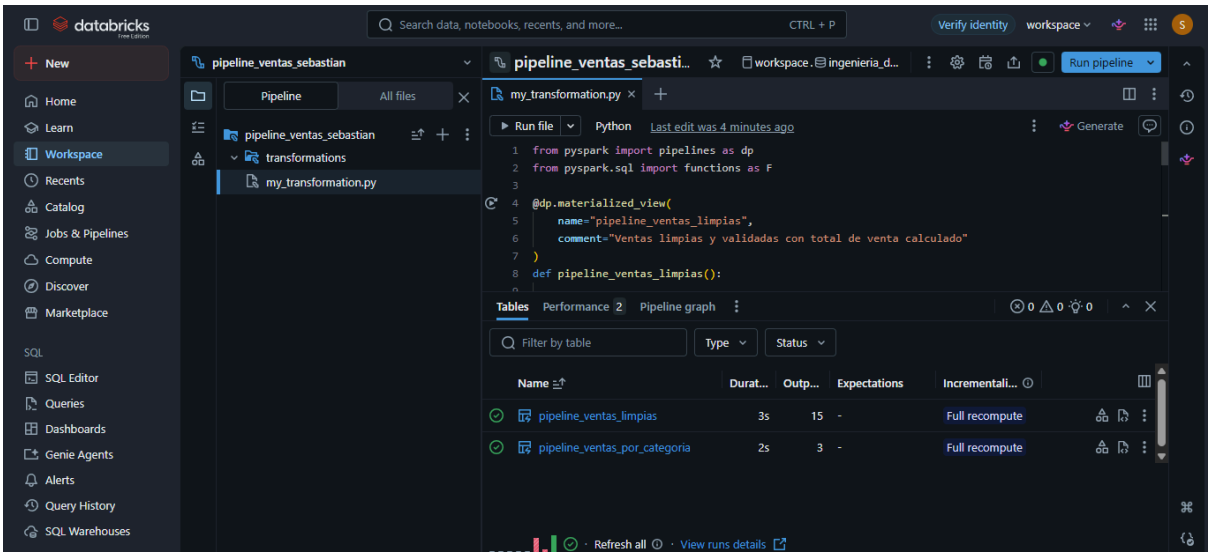
Evidencia 18. Creación y configuración del pipeline declarativo

Se creó el pipeline ETL pipeline_ventas_sebastian utilizando Lakeflow Spark Declarative Pipelines, con el catálogo workspace y el esquema ingenieria_datos_sebastian. El pipeline utiliza cómputo serverless y permite administrar las transformaciones de los datos de ventas mediante vistas materializadas.



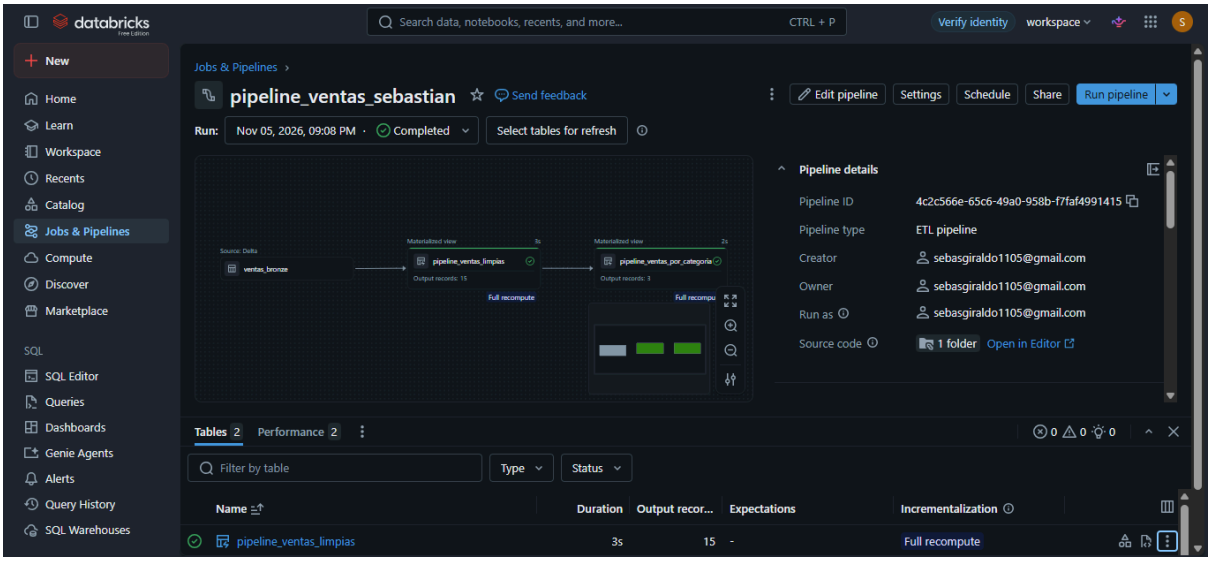
Evidencia 19. Ejecución exitosa del pipeline declarativo

Se ejecutó correctamente el pipeline declarativo, generando las vistas materializadas pipeline_ventas_limpias y pipeline_ventas_por_categoria. La primera procesó 15 registros válidos después de aplicar las reglas de limpieza, mientras que la segunda consolidó las ventas en 3 categorías. Ambas vistas finalizaron exitosamente, confirmando el funcionamiento de las transformaciones definidas.



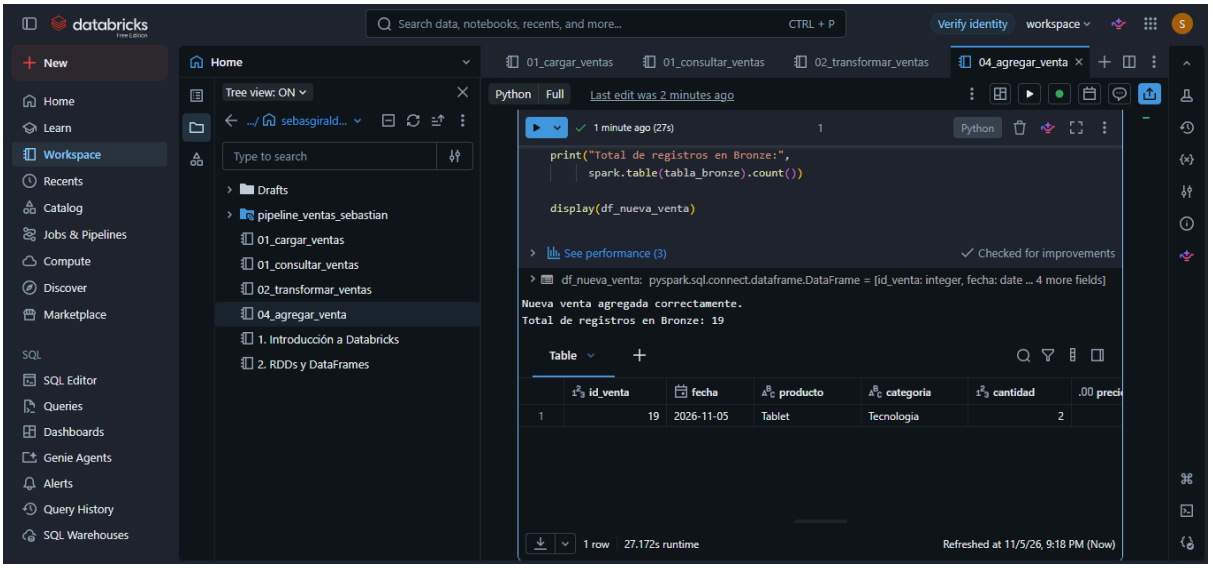
Evidencia 20. Grafo de dependencias del pipeline

Se verificó la ejecución exitosa del pipeline declarativo `pipeline_ventas_sebastian` en Databricks. El grafo muestra las dependencias entre la tabla de origen `ventas_bronze` y las vistas materializadas `pipeline_ventas_limpias` y `pipeline_ventas_por_categoria`. La ejecución procesó correctamente 15 registros limpios y generó un resumen de 3 categorías, demostrando el funcionamiento de las transformaciones y sus dependencias.



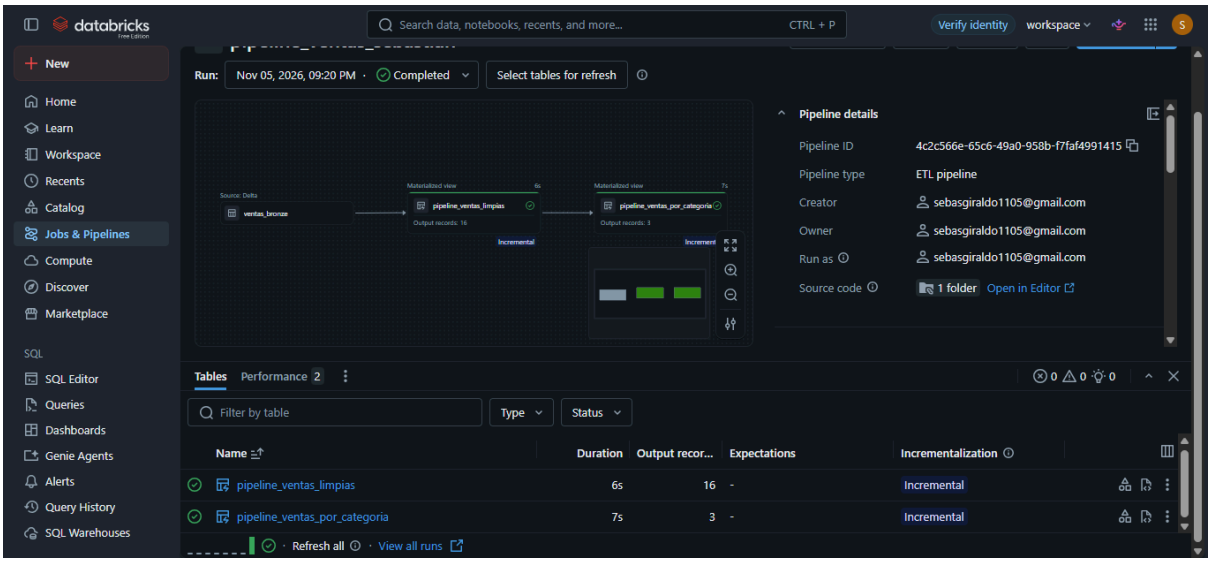
Evidencia 21. Inserción de una nueva venta en Bronze

Se agregó una nueva venta válida a la tabla Delta `ventas_bronze` mediante una operación de inserción en modo `append`, sin ejecutar nuevamente el notebook de carga inicial. El registro corresponde a una Tablet de la categoría Tecnología, con una cantidad de 2 unidades y un precio unitario de $900.000, para un valor total de $1.800.000. Esta modificación permitirá comprobar la actualización de las vistas materializadas al ejecutar nuevamente el pipeline declarativo.



Evidencia 22. Actualización incremental del pipeline

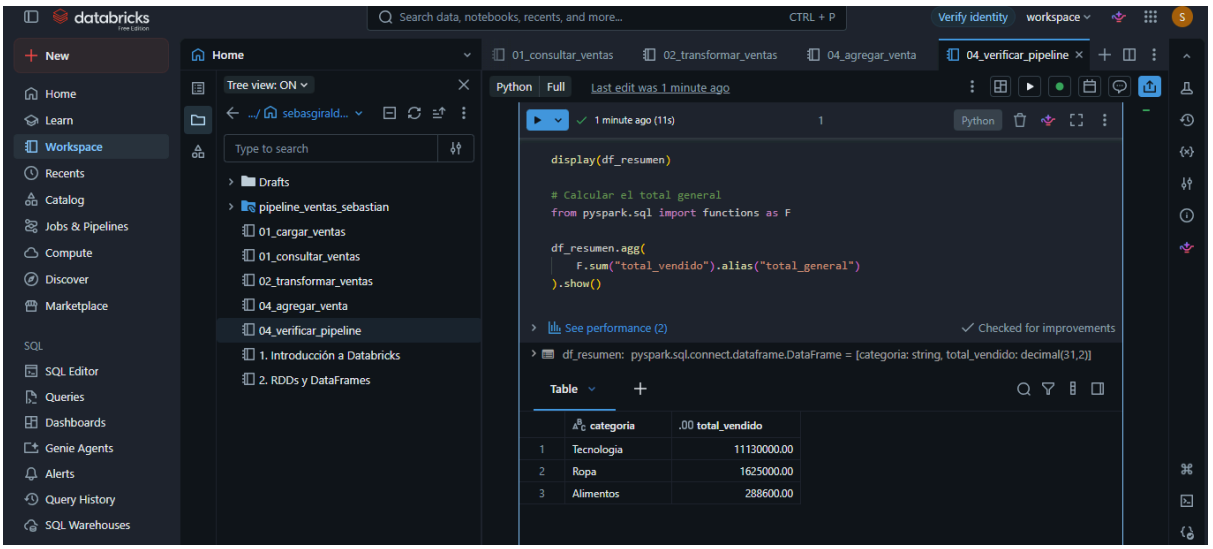
Después de insertar una nueva venta en la tabla `ventas_bronze`, se ejecutó nuevamente el pipeline declarativo `pipeline_ventas_sebastian`, sin ejecutar el notebook de carga inicial. La ejecución finalizó exitosamente y actualizó las dos vistas materializadas de manera incremental. Los registros limpios aumentaron de 15 a 16, mientras que el resumen por categoría conservó sus 3 categorías. Esto demuestra que el pipeline reconoce los cambios en los datos de origen y actualiza los resultados respetando las dependencias definidas.



Evidencia 23. Verificación del resumen actualizado

Se consultó la vista materializada `pipeline_ventas_por_categoria` después de ejecutar nuevamente el pipeline declarativo. Se verificó que el total de ventas de Tecnología aumentó de $9.330.000 a $11.130.000, representando un incremento de $1.800.000, correspondiente exactamente a la nueva venta de dos tablets.

Las categorías Ropa y Alimentos conservaron sus valores originales. El total general pasó de $11.243.600 a $13.043.600, confirmando que las transformaciones y las dependencias del pipeline funcionaron correctamente sin necesidad de ejecutar nuevamente el notebook de carga inicial.



Evidencia 24. Comparación de ventas antes y después de la actualización del pipeline declarativo

Se ejecutó una consulta en Databricks para comparar los valores de ventas por categoría antes y después de agregar una nueva venta válida a la tabla ventas_bronze.

Los resultados evidencian que la categoría Tecnología aumentó de $9.330.000 a $11.130.000, registrando una diferencia de $1.800.000, correspondiente exactamente al valor de la nueva venta. Las categorías Ropa y Alimentos permanecieron sin cambios. El total general pasó de $11.243.600 a $13.043.600.

Esta comparación confirma que el pipeline declarativo actualizó correctamente los resultados y que la nueva venta fue incorporada sin necesidad de ejecutar nuevamente el notebook de carga inicial.

